Layering, Reference Models and Encapsulation

Lecture 02

Computer Networks and Distributed Programming

The Web is an application

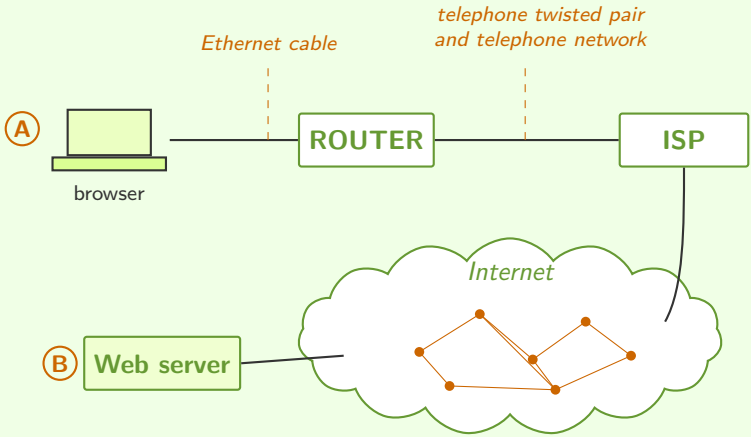
- The Web is not the Internet: it is one application that runs on top of it.
- Mail, streaming, messaging, file transfer and games are others, each asking the network for something slightly different.
- We already know what the network is made of, and how the pieces are connected.

What has to exist, and work, for one page to appear in a browser?

Today we answer that, and the answer is the shape of the rest of the course.

What one HTTP request actually crosses

A browser on A asks a page of the Web server B. Between them:



Different media, different equipment, and a mesh whose interior nobody involved knows.

Enormous complexity, and one answer

- Every box on that path is hardware plus software, built by a different vendor.
- Every segment is a different medium, with its own errors and its own speed.
- Nobody can design, or even describe, that system all at once.

Divide et impera

Break the complex system into subsystems, each with a job of its own and a well-defined way of asking the next one for help. Then design one subsystem at a time.

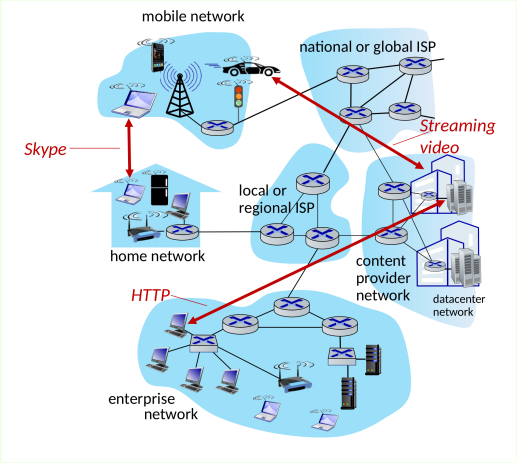
In networking that decomposition has a name: layering.

The Internet as a service infrastructure

Seen from above, the Internet is not wires: it is an infrastructure that provides services to applications — Web, video, mail, conferencing, commerce.

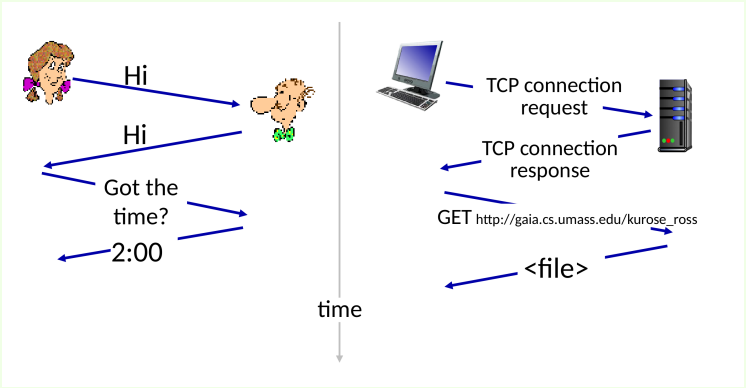
- It also provides a programming interface: the hooks an application uses to send and receive.
- Like the postal service, it offers options: fast or cheap, guaranteed or not.

Those hooks are what we will program against.



Human protocols and network protocols

A protocol fixes the format and the order of the messages, and the actions taken when one is sent or received — for people as much as for machines.



The same shape: a greeting, an answer, then the actual request

Who writes the standards

- IETF: the Internet Engineering Task Force, an open community of experts.
- It works in open working groups, each on one aspect of the Internet.
- RFC: each group publishes numbered Requests for Comments that define protocols, formats and methods — IPv4 is RFC 791, from 1981.
- ISO: the International Standards Organization, a treaty-free body of national standards organizations, works very differently: it is where OSI comes from.

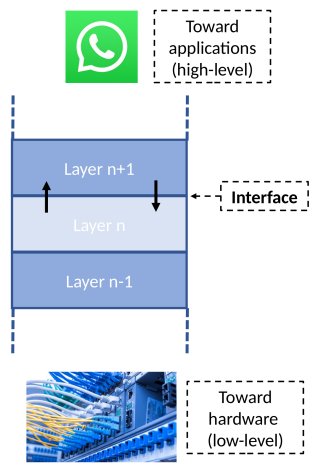
How the Internet decides

Not by committee vote but by rough consensus and running code: a specification that nobody implements does not become a standard.

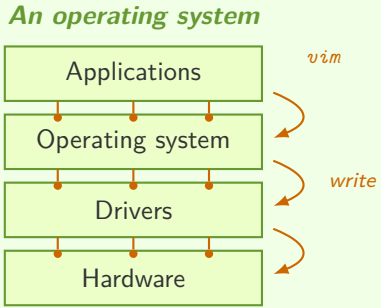
Layering

The architectural answer to complexity is a stack of layers, each one built on the one below it.

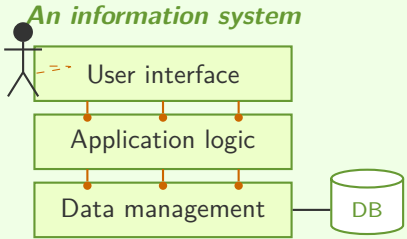
- Each layer is a level of abstraction: it hides how it does its job.
- Each layer uses the services of the layer below and provides services to the layer above.
- Between two adjacent layers there is an interface: the operations the lower one makes available.
- The top faces the applications, the bottom faces the hardware.



Layering is not a networking idea



one `write` crosses four interfaces



the user never touches the database

You already use stacks like these. The same idea appears wherever a system got too big to design in one piece — and networks are the largest case of all.

What layering buys, and what it costs

- Structure: the pieces of the system, and their relations, become nameable.
- Modularity: a layer's implementation can change and nothing else moves, as long as the service it offers stays the same.
- Replacing every telephone line with a satellite channel is a change inside one layer: the layers above never find out.

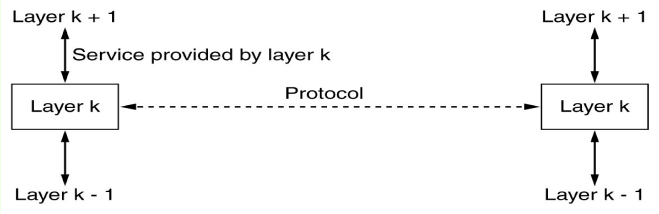
And the price

A layer may duplicate what a lower one already does — error recovery per link and end to end. And a layer sometimes needs information that lives in another one, which is exactly what layering forbids.

Service and protocol are not the same thing

Definition

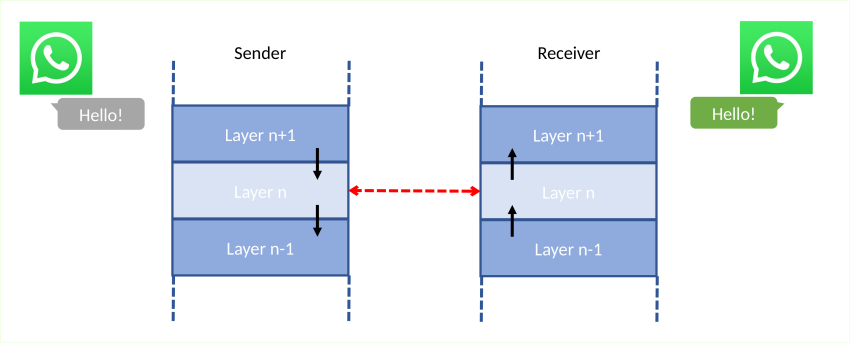
A service is the set of operations a layer offers to the layer above: it says what the layer does, never how. A protocol is the set of rules on format and meaning of the messages exchanged with the peer entity at the same layer on the other machine.



Services live on the vertical interfaces, protocols on the horizontal exchange

Entities use protocols in order to implement their service definitions.

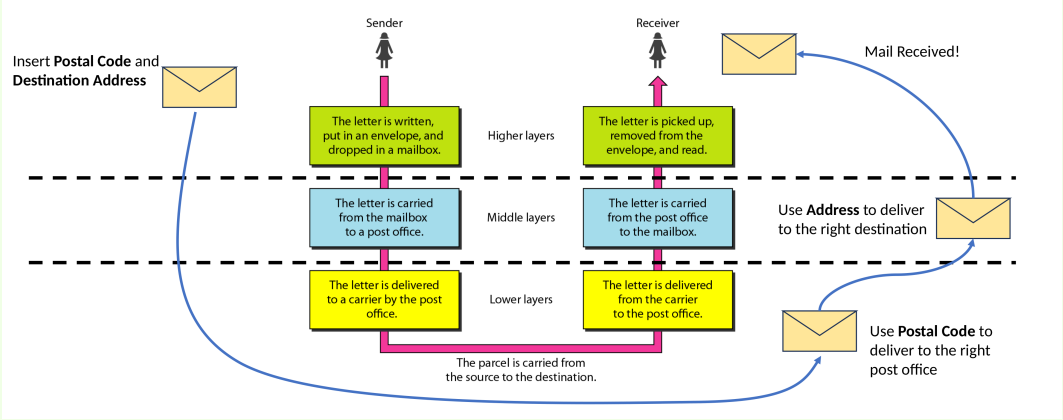
Peer layers: virtual and physical communication



- Layer *n* on the sender talks to layer *n* on the receiver, and to nothing else.
- That horizontal arrow is not a cable: it is a logical conversation.
- Data really goes down the sender's stack and up the receiver's. Only at the physical layer does anything actually travel.

The postal analogy

Each layer of the postal system does one job and hands the letter down; the address is read by the layer it was written for, and by no other.



Connection-oriented service

Modelled on the telephone system: you establish, you use, you release.

- The connection behaves like a tube: what goes in one end comes out the other, normally in the same order.
- Establishing it allows a negotiation — maximum message size, quality of service, and so on: one side proposes, the other accepts or counters.
- A connection that also reserves resources, such as a fixed rate, is a circuit.

And if you have one short message to send, is all that worth it?

Connectionless service

Modelled on the postal system: nothing is established before sending.

- Each message carries the full destination address and is handled individually by the intermediate nodes.
- Between the same two ends the mesh offers several paths, so two messages may take different ones — and may arrive out of order.
- Store-and-forward: the node receives a message in full before sending it on.
- Cut-through: the node starts sending it on before it has finished receiving it.

Datagram

An unreliable — that is, unacknowledged — connectionless service, named after the telegram, which also came back with no receipt.

Reliability, and why you would ever give it up

Reliable means the receiver acknowledges what it got, so the sender knows it arrived. Both kinds of service can be reliable or not.

- Acknowledgements cost overhead and delay, and sometimes the price is too high.
- Reliability may not be on offer: Ethernet does not provide it, and higher layers are left to recover.
- For voice over IP, a moment of noise is better than a gap waiting for a retransmission — so unreliable is the right choice, not a compromise.

Most reliable services in this course are built on top of an unreliable one.

Six kinds of service

	Service	Example
Connection-oriented	Reliable message stream	Sequence of pages
	Reliable byte stream	Movie download
	Unreliable connection	Voice over IP
Connection-less	Unreliable datagram	Electronic junk mail
	Acknowledged datagram	Text messaging
	Request-reply	Database query

Two rows are worth remembering: reliable byte stream is what TCP will give us, unreliable datagram is what UDP will give us.

Service primitives

A service is specified formally by the primitives through which a process asks for it. When the stack lives in the operating system, they are system calls.

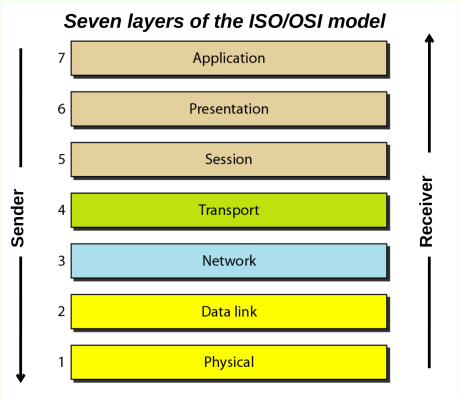
Primitive	Meaning
`LISTEN`	block waiting for an incoming connection
`CONNECT`	establish a connection with a waiting peer
`ACCEPT`	accept an incoming connection from a peer
`RECEIVE`	block waiting for an incoming message
`SEND`	send a message to the peer
`DISCONNECT`	terminate the connection

Look at them closely: this is similar to the interface you will be writing C against.

The OSI reference model

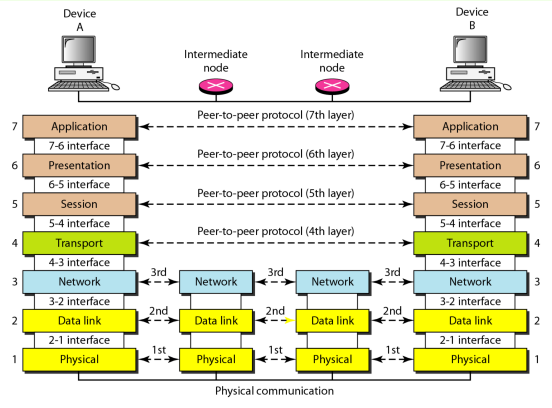
In the 1980s the ISO defined a layered reference model for computer networks: OSI, Open Systems Interconnection — systems open to communicating with other systems.

- Seven layers, each created where a different abstraction was needed.
- Its biggest contribution is making services, interfaces and protocols three explicitly separate things.



Not every node implements every layer

- Only the end systems implement the whole stack up to the application.
- Intermediate nodes — switches, routers — implement only two or three layers.
- Nothing above the network layer: a router has no business reading your message.

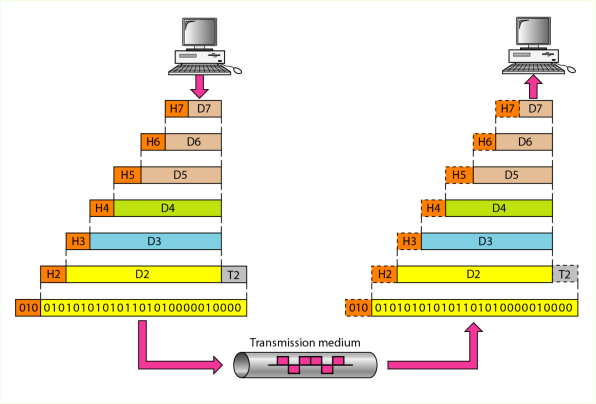


Encapsulation and decapsulation

Going down: each layer of the sender adds its own field to what it received — a header in front, sometimes a trailer behind.

Going up: each layer of the receiver removes and interprets exactly the field its peer added, and passes the rest upward.

So a unit at any layer is always header plus payload, and the payload is the unit from the layer above.



Below its own layer, a payload is opaque

When the layer-7 message goes down, everything that came from above — header and content together — becomes the payload of layer 6.

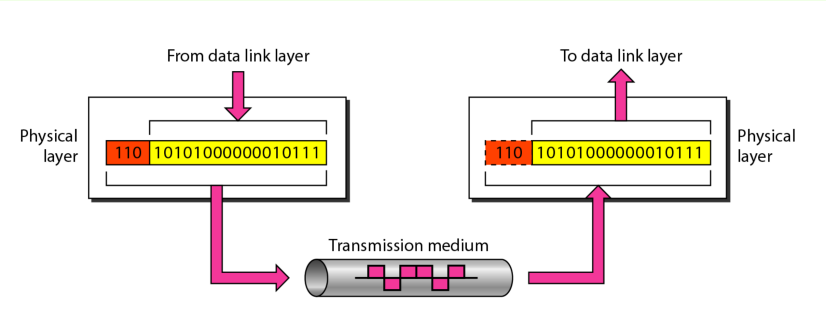
- Layer 6 prepends its own header and hands the whole thing to layer 5, which does it again.
- Layer 6 does not know it is carrying HTML, and must not care: for it the payload is an opaque sequence of bytes.
- That is precisely what makes the layers independent. A layer that inspected its payload would break the moment the layer above changed.

The receiver walks the same sequence backwards, each layer stripping its own header.

OSI 1: the physical layer

Responsible for moving individual bits from one node to the next one along the wire.

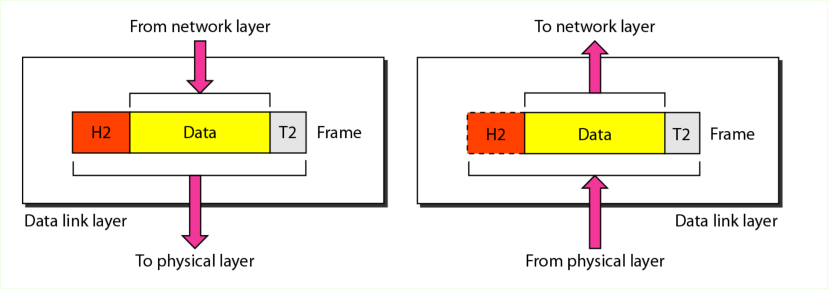
- Connectors, cables, electrical signals; point-to-point or multipoint lines.
- Representation of bits, data rate, and bit synchronisation between the two ends.



OSI 2: the data link layer

Responsible for moving frames from one node to the next one: one hop, not the whole path.

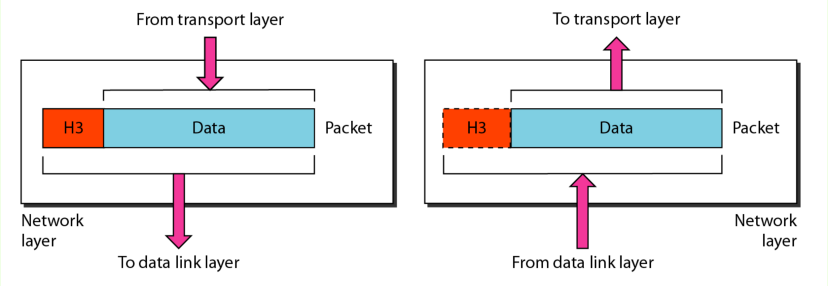
- A frame is a bounded piece of data, typically a few hundred bytes.
- Flow control, error control through frame check sequences, and access control when the medium is shared.



OSI 3: the network layer

Responsible for delivering packets from the source host to the destination host, across the whole path.

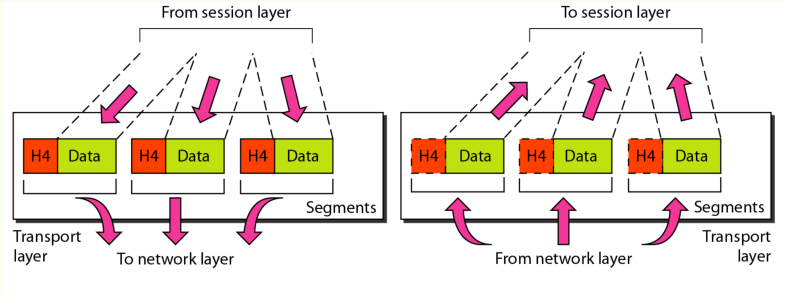
- It is the layer that has to get a packet through the topology: a graph of nodes with more than one way across.
- Logical addressing, and routing — deciding, node by node, which way out to take.



OSI 4: the transport layer

Responsible for delivering a message end to end, and to the right program on the destination host.

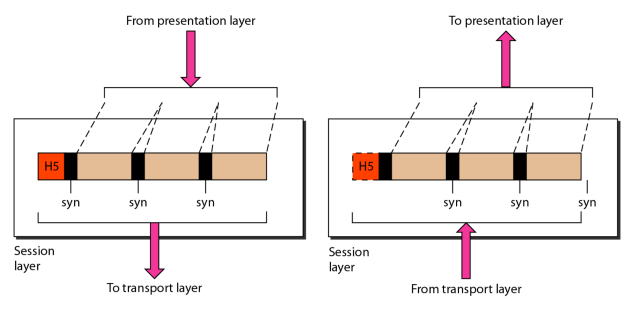
- Port addressing: the network layer finds the host, this layer finds the process.
- Connection control, connection-oriented or connectionless.
- Segmentation and reassembly, flow control and error control.



OSI 5: the session layer

Responsible for dialog control: establishing, maintaining and synchronising the interaction between the two systems.

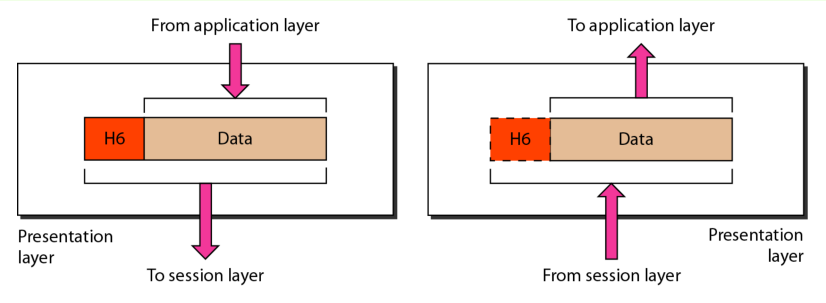
- Who speaks when, and for how long.
- Checkpoints, so that an interrupted exchange resumes from the last one instead of from the beginning.



OSI 6: the presentation layer

Responsible for the representation of the data, so that what one side writes is what the other side reads.

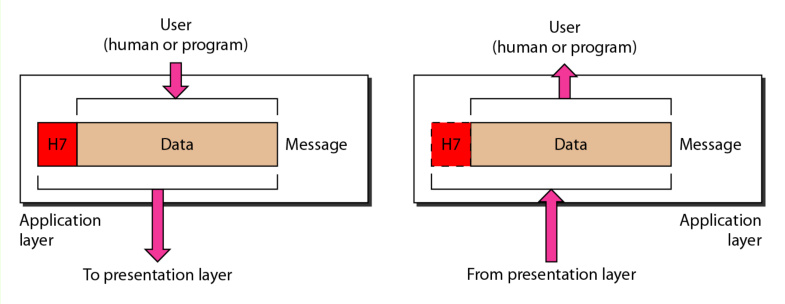
- Translation between encodings, ASCII against EBCDIC being the historical case.
- Encryption and decryption; compression.



OSI 7: the application layer

Responsible for providing services to the user, and it is where the protocols you already use by name live.

- Remote login, file transfer and access, mail, the Web.
- This is the layer we will open first, because it is the one you can see.

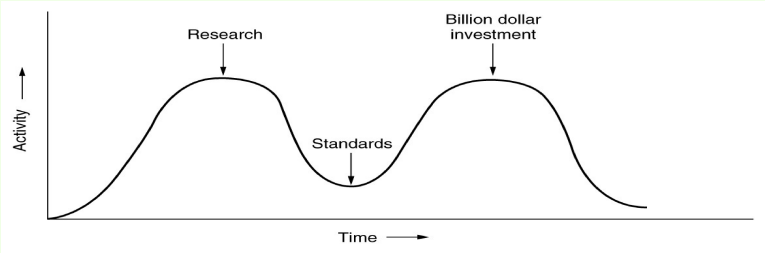


Why OSI did not win

- Bad timing: TCP/IP was already deployed in research universities when the OSI protocols appeared, and no vendor wanted a second stack.
- Bad design: seven layers was a political number: session and presentation are nearly empty, data link and network are overfull.
- Bad implementations: the first ones were huge, slow and unwieldy, and the reputation stuck long after the products improved.
- Bad politics: it was seen as imposed by governments and ministries, while TCP/IP arrived free with Berkeley UNIX.

The model survived and is still how we talk about networks. The protocols did not.

The apocalypse of the two elephants



Research first, then the billion-dollar investment

A standard written too early freezes a subject nobody understands yet; one written too late is ignored by everyone who has already invested. It has to land in the trough — and OSI did not.

What is wrong with the TCP/IP model

- It does not clearly separate service, interface and protocol — the one thing OSI got right.
- It is not general: try describing Bluetooth with it and you cannot.
- Its link layer is not a layer at all, but an interface between the host and the transmission link.
- It does not separate physical from data link, which are two completely different jobs.

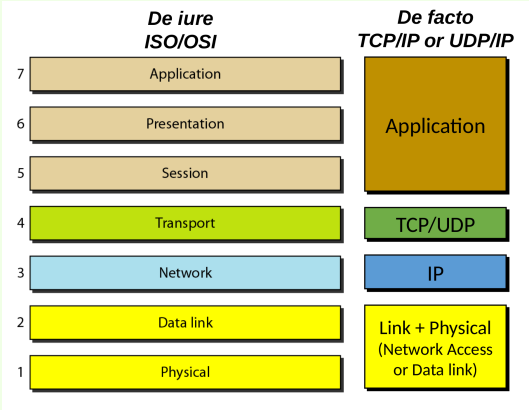
Its protocols, though, were carefully designed, well implemented, and given away.

De iure and de facto

De iure: ISO/OSI is the standard model for computer networks: complete, detailed, and complex.

De facto: the protocols actually running on the Internet and on local networks are TCP, UDP and IP.

The two are not rivals in practice: we keep the model to think with and the protocols to work with.



The five layers we will use from here on

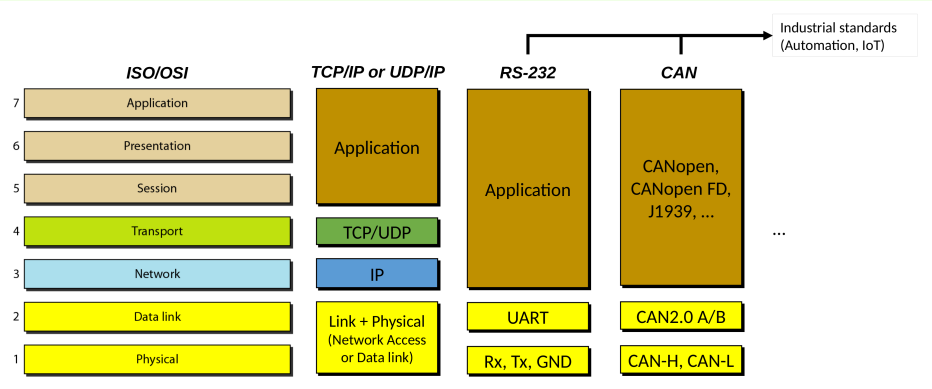
From this point the course counts five layers, not seven: the OSI model minus session and presentation, whose few real jobs the applications do themselves.

Layer	Corresponds to OSI	Unit of data
Application	7, 6 and 5	message
Transport	4	segment
Network	3	datagram
Link	2	frame
Physical	1	bit

We keep the OSI numbers when we need them — layer 3, layer 2 — because everyone in the field uses them. But the stack we study, top down, is this one.

Layering beyond the Internet

Other stacks exist and are chosen for the job: RS-232 for a serial line, CAN inside a vehicle. They are far simpler — and they are still layered.



Summary

- One request crosses too many pieces to design at once, so the system is cut into layers: divide et impera.
- A layer offers a service upward through an interface, and runs a protocol sideways with its peer. The two are independent.
- Peer communication is logical: data goes down one stack and up the other, and only the physical layer moves anything.
- Encapsulation is what makes that work — each layer treats what it carries as opaque bytes.
- OSI gave us seven layers to think with, TCP/IP gave us the protocols; we use five layers and Kurose's names.

Next we open the top of the stack, where the applications are.